

GPU-Accelerated Cryptography

SHA-256 Hashing

Technical Architecture & Performance Report Version 2.0 · June 19, 2026

Author(s)

Ankesh Kumar

Postgraduate Diploma in Artificial Intelligence
Indian Institute of Technology Jodhpur
Jodhpur, India
Email address: g25ait1022@iitj.ac.in

Vijender Singh Thakur

Postgraduate Diploma in Artificial Intelligence
Indian Institute of Technology Jodhpur
Jodhpur, India
Email address: g25ait1188@iitj.ac.in

Anudeep S

Postgraduate Diploma in Artificial Intelligence
Indian Institute of Technology Jodhpur
Jodhpur, India
Email address: g25ait1026@iitj.ac.in

Nishtha Parashar

Postgraduate Diploma in Artificial Intelligence
Indian Institute of Technology Jodhpur
Jodhpur, India
Email address: g25ait1132@iitj.ac.in

1. Objective

The objective of this implementation is to evaluate the performance of a CUDA-based parallel implementation of the **SHA-256** cryptographic hash algorithm on different NVIDIA GPU architectures and compare it with the corresponding CPU implementation. A batch of 1,000,000 identical input strings was processed to demonstrate the benefit of massive parallel execution.

2. SHA-256 Algorithm Overview

SHA-256 (Secure Hash Algorithm 256-bit) is a member of the SHA-2 family standardized by NIST in FIPS 180-4. It produces a 256-bit (32-byte) digest from an arbitrary-length input.

2.1 Algorithm Structure

SHA-256 processes input in 512-bit (64-byte) blocks through a Merkle–Damgård construction:

1. Message Padding — append 0x80, zero bytes, and 64-bit big-endian message length to make the total length a multiple of 512 bits.
2. Block Splitting — divide padded message into 512-bit blocks.
3. Message Schedule Expansion — expand each 16-word block into 64 32-bit words ($W[0..63]$) using mixing functions σ_0 and σ_1 .
4. Compression Function — 64 rounds operating on eight 32-bit state words (a, b, c, d, e, f, g, h) using rotation and XOR operations.
5. Hash Accumulation — add compressed output to running hash state after each block.
6. Output — concatenate final eight state words as 256-bit digest.

2.2 Core Operations

Every round uses these bitwise primitives — all computable in a single GPU clock cycle:

Operation	Formula	Description
$\text{ROTR}(x,n)$	$(x \gg n) \mid (x \ll 32-n)$	Rotate right — lossless bit rotation
$\text{CH}(e,f,g)$	$(e \& f) \wedge (\sim e \& g)$	Choose — select f or g based on e
$\text{MAJ}(a,b,c)$	$(a\&b) \wedge (a\&c) \wedge (b\&c)$	Majority vote of three bits
$\Sigma_0(a)$	$\text{ROTR}(2) \wedge \text{ROTR}(13) \wedge \text{ROTR}(22)$	Upper sigma — mixes state word a
$\Sigma_1(e)$	$\text{ROTR}(6) \wedge \text{ROTR}(11) \wedge \text{ROTR}(25)$	Lower sigma — mixes state word e
$\sigma_0(w)$	$\text{ROTR}(7) \wedge \text{ROTR}(18) \wedge \text{SHR}(3)$	Small sigma 0 — schedule mixing
$\sigma_1(w)$	$\text{ROTR}(17) \wedge \text{ROTR}(19) \wedge \text{SHR}(10)$	Small sigma 1 — schedule mixing

2.3 Why SHA-256 is GPU-Friendly

Three properties make SHA-256 an ideal GPU workload:

- No data dependencies across messages — thread N never reads thread M's data
- Fixed, predictable memory access pattern — no dynamic pointer chasing or branch divergence between messages
- High arithmetic intensity — 64 rounds of computation per 64-byte input block; keeps GPU ALUs busy relative to memory bandwidth consumed.

3. Innovation

The proposed implementation exploits the parallel processing capability of CUDA-enabled GPUs by assigning each SHA-256 computation to an independent GPU thread. Since every hash computation is independent, SHA-256 is an ideal candidate for data-level parallelism.

The implementation demonstrates:

- Parallel execution of one million independent SHA-256 computations.
- Comparison between CPU and GPU execution on different GPU architectures.
- Performance scalability across entry-level (MX250), mid-range (GTX 1650), and modern RTX (RTX 4060 Laptop) GPUs.
- Verification that the **original SHA-256 digest remains identical** across all executions, confirming the deterministic nature of the SHA-256 algorithm.

Although intermediate hashes (#250000, #500000, etc.) differ because each iteration hashes the previous digest (hash chaining), the computation is completely deterministic and reproducible.

4. Experimental Setup

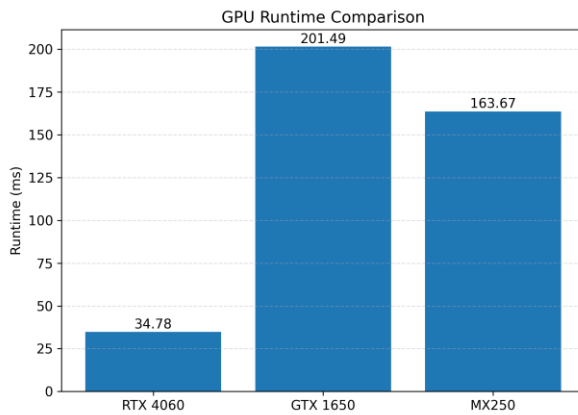
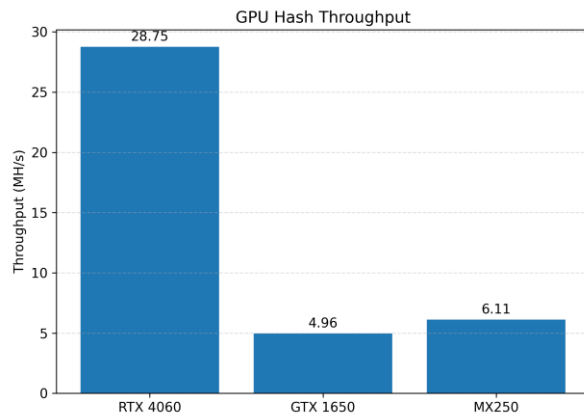
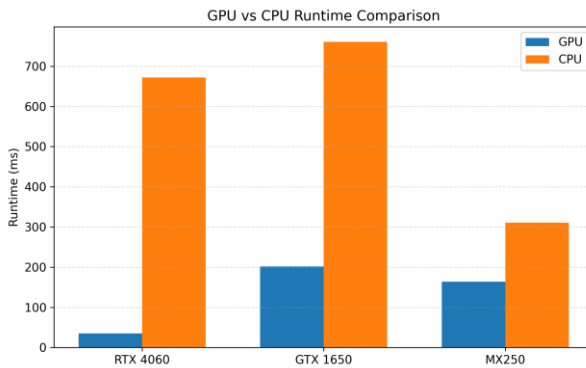
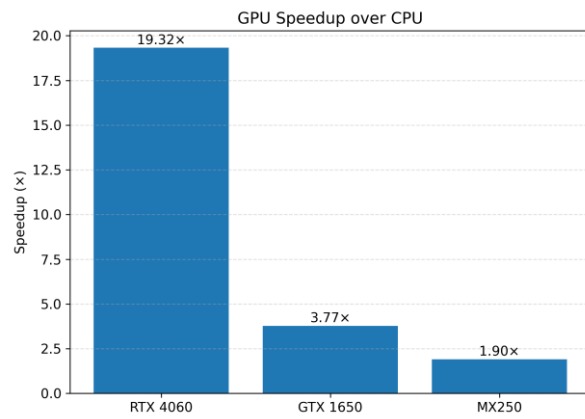
- **Input:** "abc"
- **Batch Size:** 1,000,000 hashes
- **Algorithm:** SHA-256
- **Implementation:** CUDA C++
- **Platforms Tested:**
 - NVIDIA GeForce RTX 4060 Laptop GPU
 - NVIDIA GeForce GTX 1650
 - NVIDIA GeForce MX250

5. Results & Discussion

The experimental evaluation was conducted to assess the effectiveness of the proposed GPU-accelerated SHA-256 implementation across different NVIDIA GPU architectures. Performance was measured using execution time, hashing throughput, and GPU speedup over the CPU for a batch of one million hash computations. The quantitative results are summarized in **Table 1**, while **Figures 1(a)–1(d)** provide a visual comparison of the obtained performance metrics.

Table 1. Performance Comparison

GPU	CUDA Core	Memory	GPU Runtime (ms)	GPU Throughput (MH/s)	CPU Runtime (ms)	GPU Speedup
RTX 4060 Laptop	3072	8 GB GDDR6	34.78	28.75	671.90	19.3×
GTX 1650	896	4 GB GDDR5	201.49	4.96	760.49	3.77×
MX250	384	2 GB GDDR5	163.67	6.11	310.79	1.90×

**(a)****(b)****(c)****(d)****Figure 1(a)** GPU Runtime Comparison **1(b)** Hash Throughput Comparison **1(c)** GPU vs CPU Runtime Comparison **1(d)** GPU Speedup over CPU

The experimental results clearly demonstrate that GPU architecture significantly affects SHA-256 performance.

- The RTX 4060 GPU achieved the highest throughput (28.75 MH/s) owing to its larger number of CUDA cores, newer Ada Lovelace architecture, and faster GDDR6 memory.
- The GTX 1650 provided moderate acceleration over the CPU, delivering approximately 3.8× speedup.

- Although the MX250 contains fewer CUDA cores, it still outperformed the CPU, confirming that even entry-level GPUs benefit from parallel cryptographic workloads.
- Across all systems, the SHA-256 digest for the original input remained identical, demonstrating the correctness and determinism of the implementation.

6. Conclusion

The CUDA-based implementation successfully accelerates SHA-256 by leveraging thousands of GPU threads to compute hashes concurrently. The experiments confirm that GPU acceleration substantially reduces execution time for large batches of independent hash computations, with the **RTX 4060** achieving the best performance at **28.75 MH/s** and approximately **19× faster** execution than the CPU. These results highlight the effectiveness of modern GPUs for high-throughput cryptographic applications such as blockchain, password hashing, digital forensics, and large-scale integrity verification.

7. References & Standards

- NIST FIPS 180-4, "Secure Hash Standard (SHS)," August 2015.
- RFC 6234, "US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)," May 2011.
- NVIDIA CUDA C++ Programming Guide, v12.x. developer.nvidia.com/cuda-documentation
- NVIDIA PTX ISA Reference 8.x — `byte_perm`, rotation intrinsics.
- OpenCL 3.0 Specification, Khronos Group, 2020. [khronos.org/opencl](https://www.khronos.org/opencl)
- Bernstein & Lange, "Non-uniform caches," Fast Software Encryption 2007.
- NVIDIA Ampere Architecture Whitepaper, "NVIDIA A100 Tensor Core GPU Architecture," 2020.